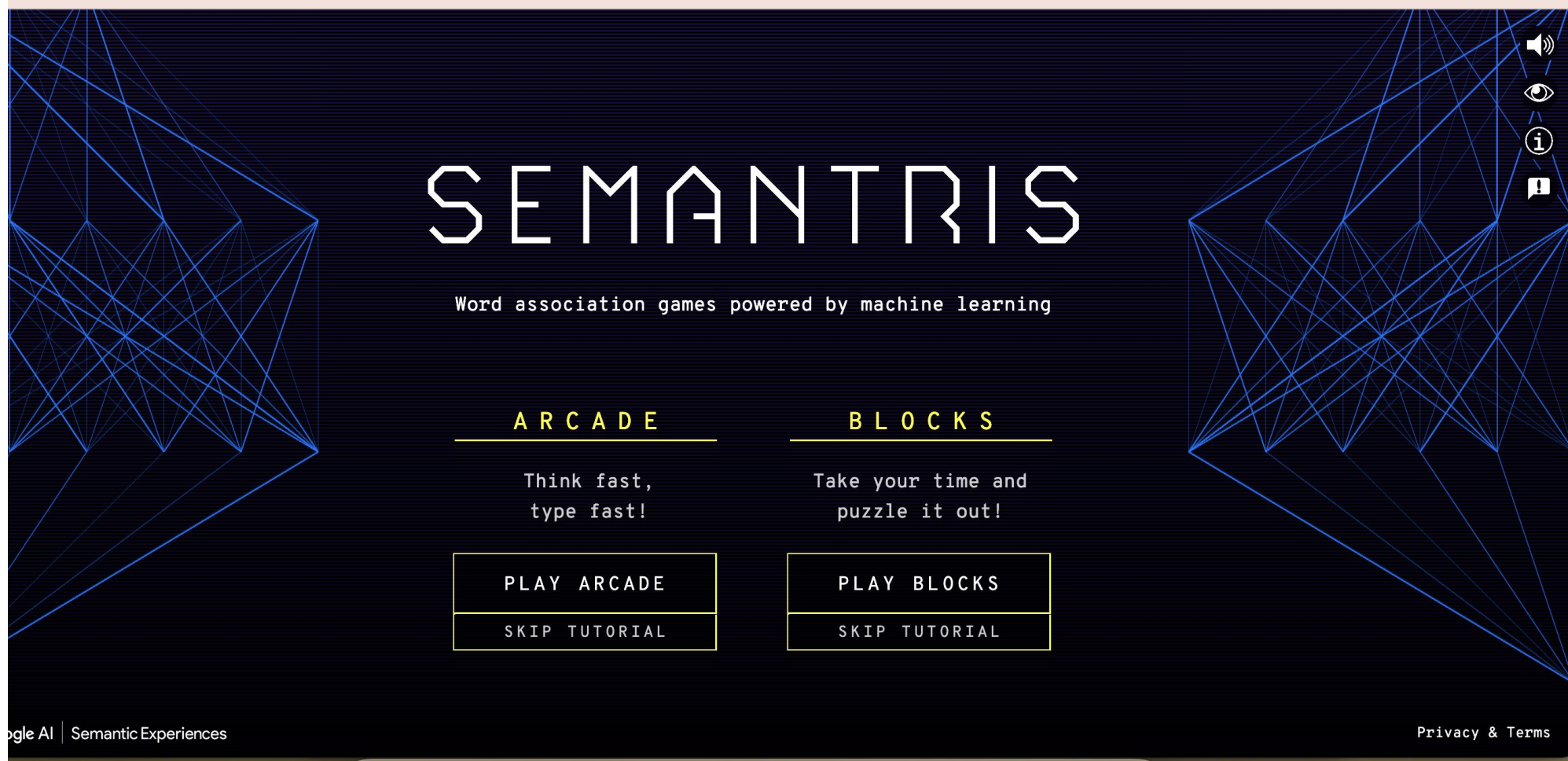


《神经网络与深度学习》

第九章—无监督学习

人工智能学院

王亚星



<https://research.google.com/semantis/>

◆ 无监督学习

◆ 聚类

- ◆ K-means

- ◆ 层次式聚类

◆ 特征学习

- ◆ 主成分分析

- ◆ 稀疏编码

- ◆ 自编码器

- ◆ 稀疏自编码器

- ◆ 降噪自编码器

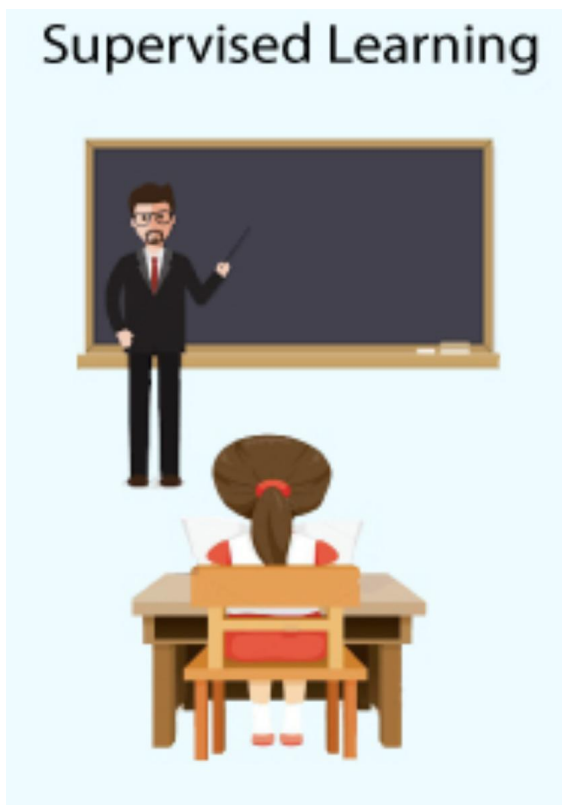
◆ 概率密度估计

- ◆ ~~参数密度估计~~

- ◆ 非参数密度估计

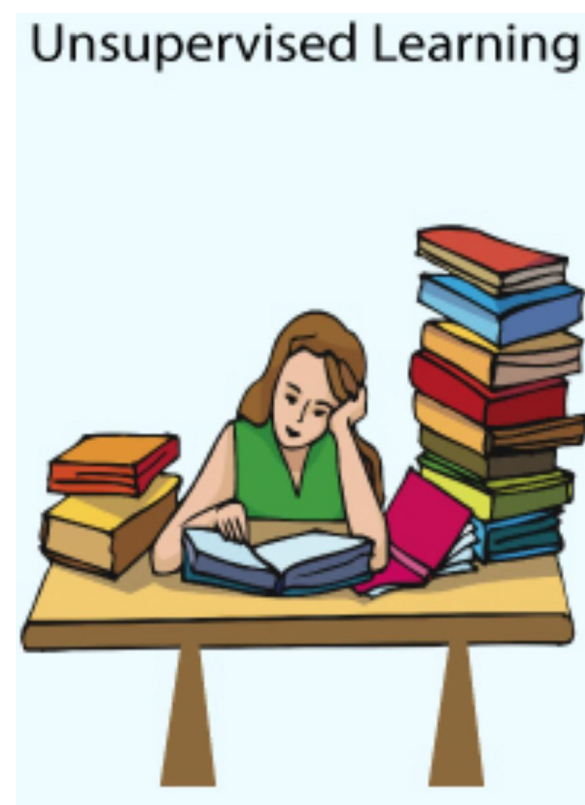
◆ 监督学习

◆ 建立映射关系 $f: x \rightarrow y$



◆ 无监督学习

◆ 从无标签的数据中学习出一些有用的模式。

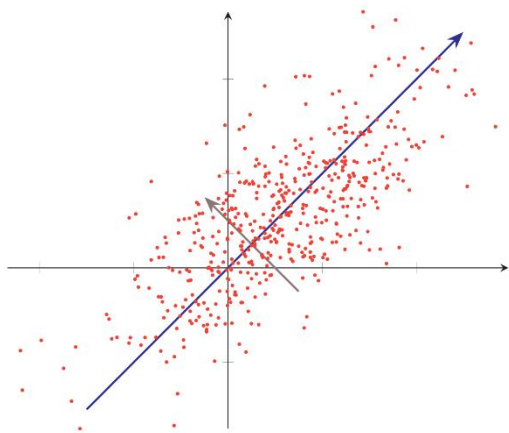


为什么要无监督学习？

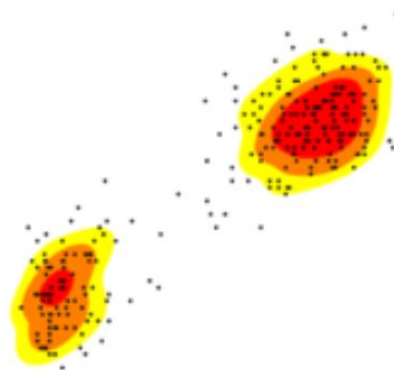
大脑有大约 10^{14} 个突触，我们只能活大约 10^9 秒。所以我们有比数据更多的参数。这启发了我们必须进行大量无监督学习的想法，因为感知输入（包括本体感受）是我们可以获得每秒 10^5 维约束的唯一途径。

—— Geoffrey Hinton, 2014 AMA on Reddit

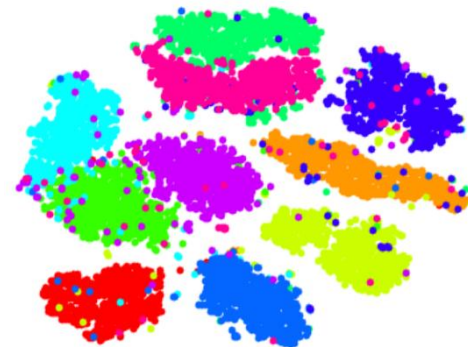
◆典型的无监督问题



无监督特征学习



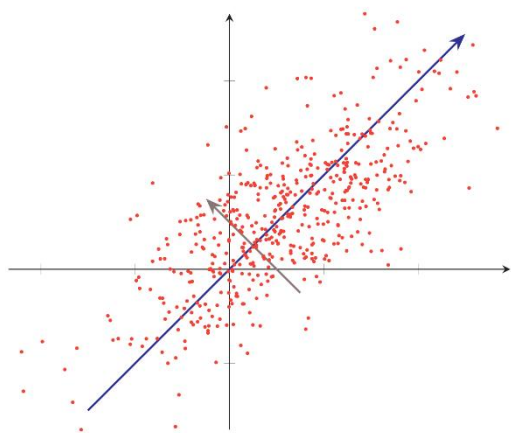
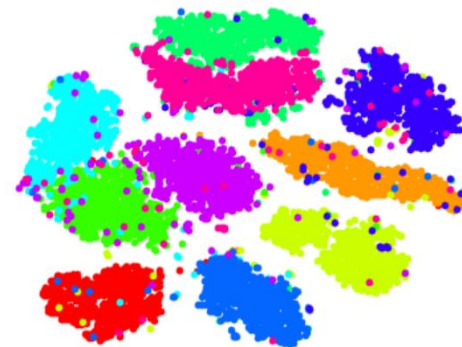
密度估计



聚类

◆ 典型的无监督问题

聚类

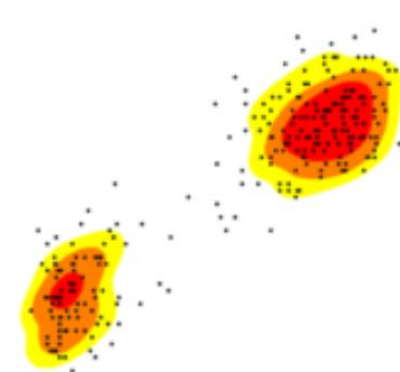


特征学习

x_1	x_2	x_3	$\dots$	x_n

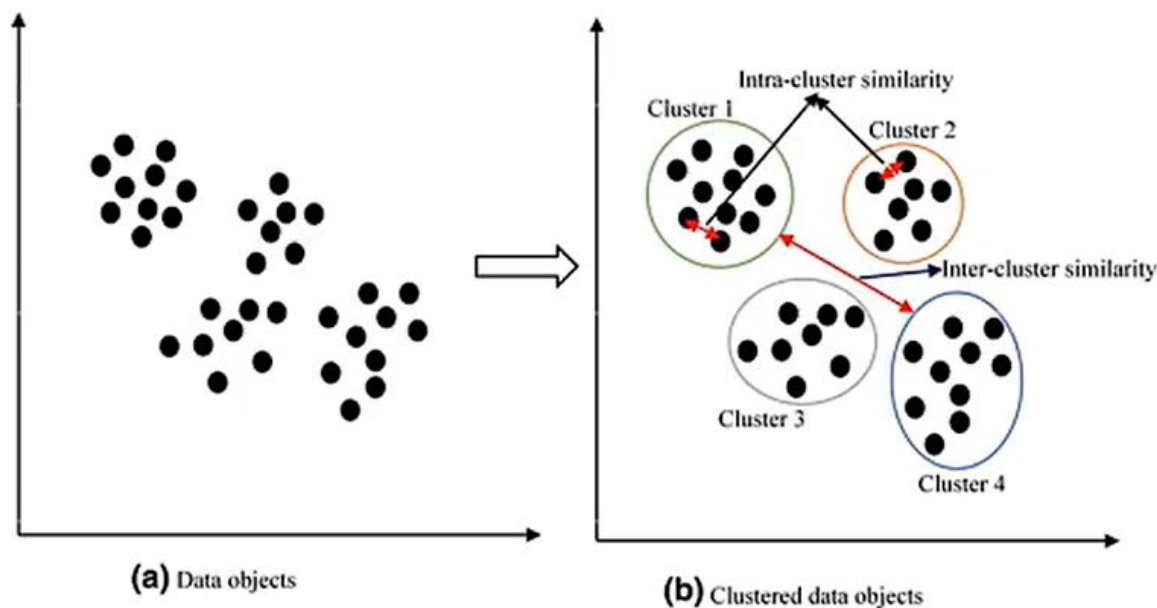
数据集

密度估计



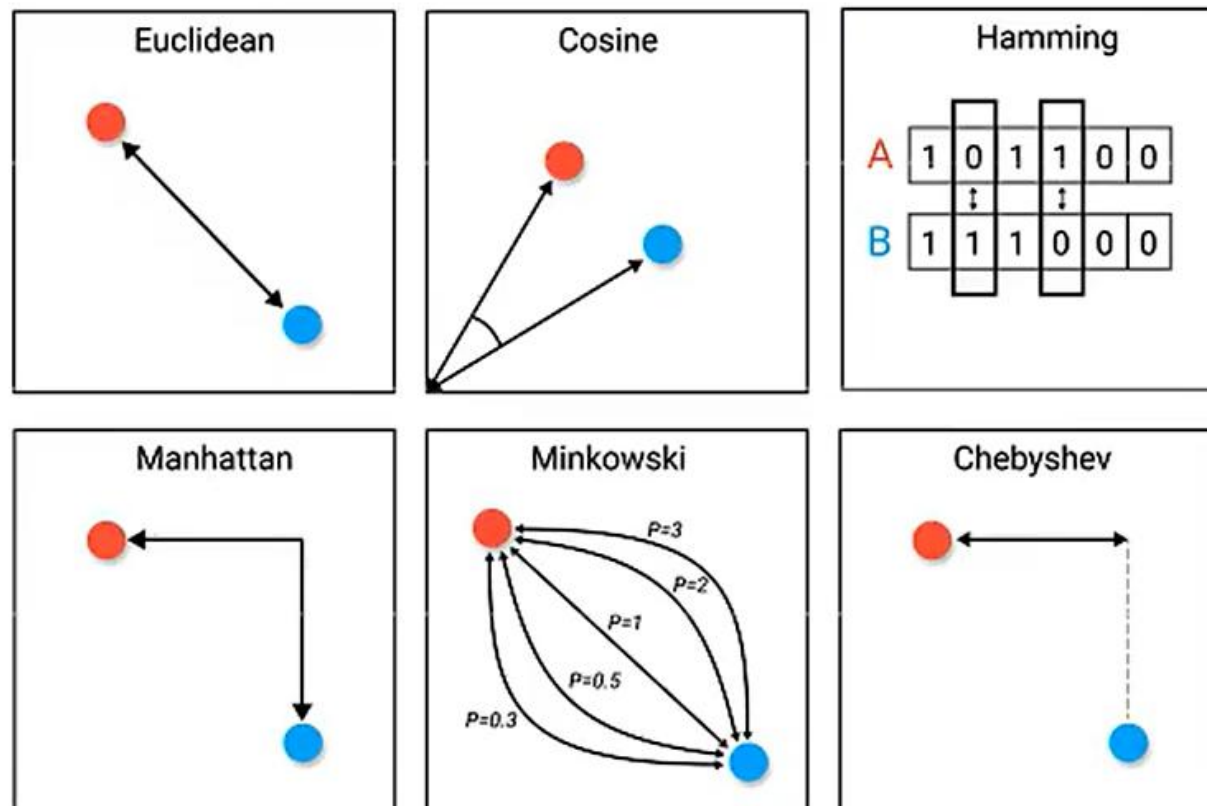
聚类

◆ 将样本集合中相似的样本分配到相同的类或者簇中，不相似的样本分配到不同的类或者簇，使得类内样本间距较小，类间样本间距较大。



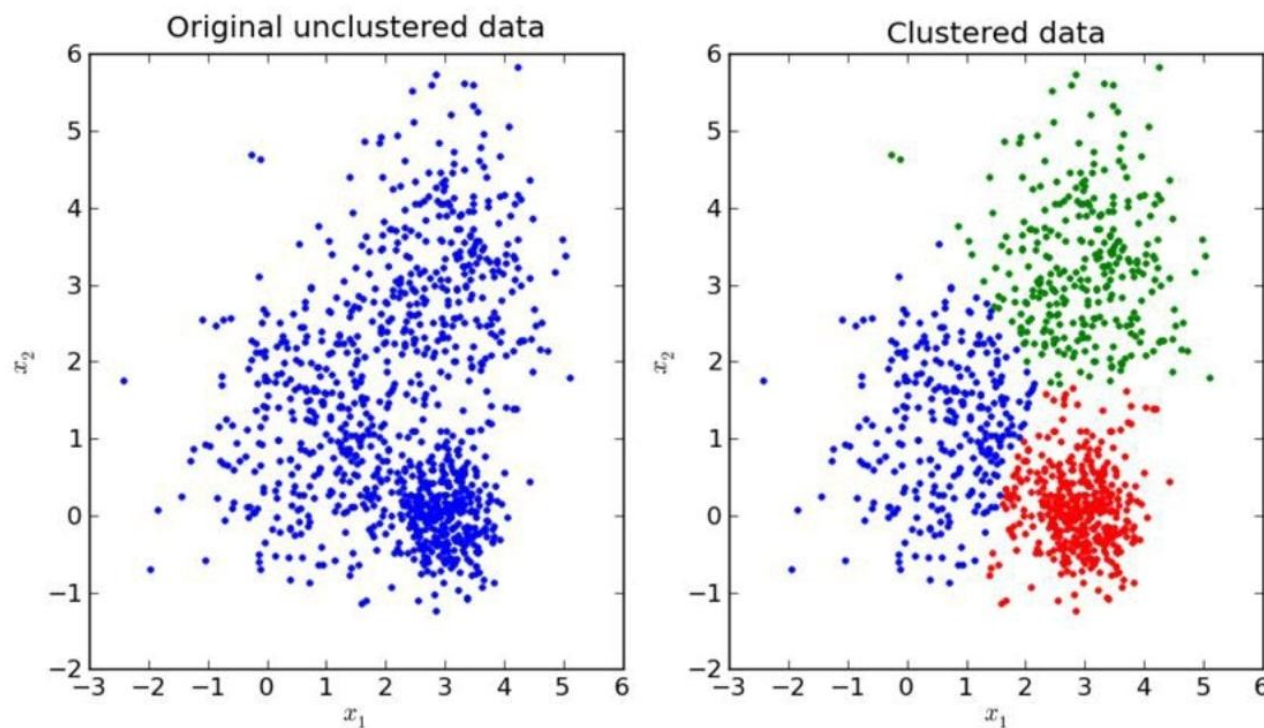
◆ 样本相似性（距离度量）

- ◆ L1、L2 距离
- ◆ 余弦距离
- ◆ 曼哈顿距离
- ◆ 相关系数
- ◆ 汉明距离



◆ 类 / 簇

◆ 没有严格定义



理解不同
结果不同

◆ 类内距离

◆ 样本间平均距离

$$avg(C) = \frac{2}{|C|(|C|-1)} \sum_{1 \leq i < j \leq |C|} d_{ij}$$

类内所有样本间
距离的平均

◆ 样本间最大距离

$$dia(C) = \max_{1 \leq i < j \leq |C|} d_{ij}$$

类内所有样本间
的最大距离

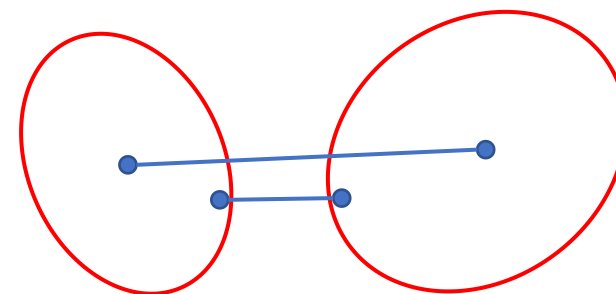
◆ 类间距离

◆ 样本间最短距离

$$D_{pq} = \min\{d_{ij} | x^{(i)} \in C_p, x^{(j)} \in C_q\}$$

◆ 样本均值间距离

$$D_{pq} = d_{\mu_p \mu_q}$$



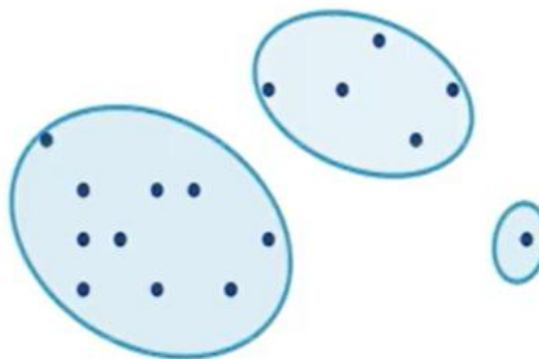
◆ K-means 聚类

◆ 层次聚类

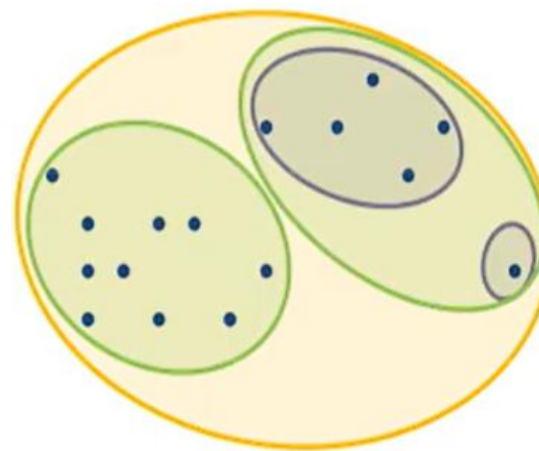
◆ 密度聚类

◆ 谱聚类

Partitional Clustering



Hierarchical Clustering



外部指标

有外部可参考的聚类结果

- ◆ Jaccard 系数
- ◆ FM 指数
- ◆ Rand 指数

标签
不对应

混淆矩阵		预测结果	
		同类	不同类
真实结果	同类	#TP	#FN
	不同类	#FP	#TN

数据集中每组数据($x^{(i)}, x^{(j)}$)
在预测/参考中结果

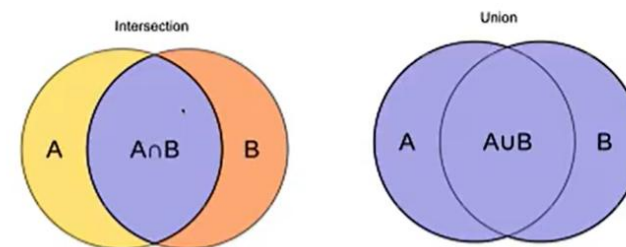
内部指标

无外部可参考的聚类结果

- ◆ DB 指数
- ◆ Dunn 指数

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{avg(C_i) + avg(C_j)}{d_{\mu_i \mu_j}} \right)$$

类内距离 ↓
类间距离 ↑



$$JC = \frac{\#TP}{\#TP + \#FN + \#FP}$$

Jaccard 系数

K-means 聚类

[Blog](#) [About](#) [Contact](#) [I'm Feeling Lucky](#)

Visualizing K-Means Clustering

January 19, 2014

Suppose you plotted the screen width and height of all the devices accessing this website. You'd probably find that the points form three clumps: one clump with small dimensions, (smartphones), one with moderate dimensions, (tablets), and one with large dimensions, (laptops and desktops). Getting an algorithm to recognize these clumps of points without help is called *clustering*. To gain insight into how common clustering techniques work (and don't work), I've been making some visualizations that illustrate three fundamentally different approaches. This post, the first in this series of three, covers the k-means algorithm. To begin, click an initialization strategy below:

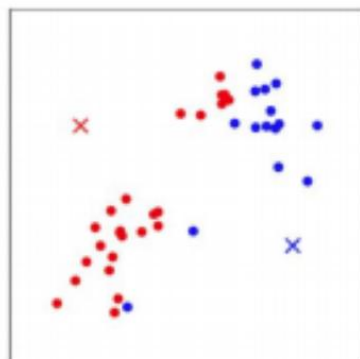
<https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>



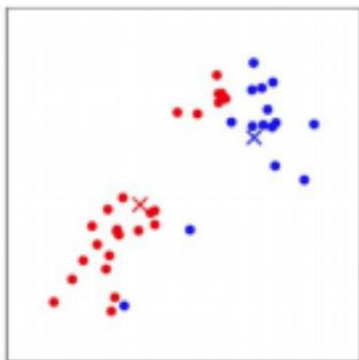
(a)



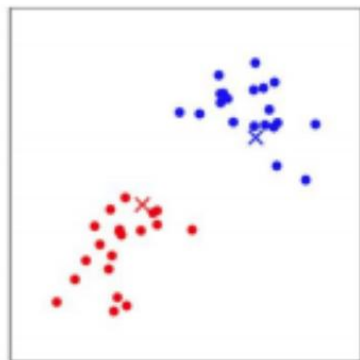
(b)



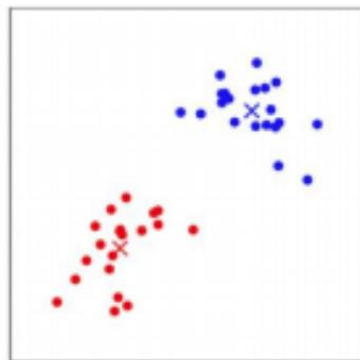
(c)



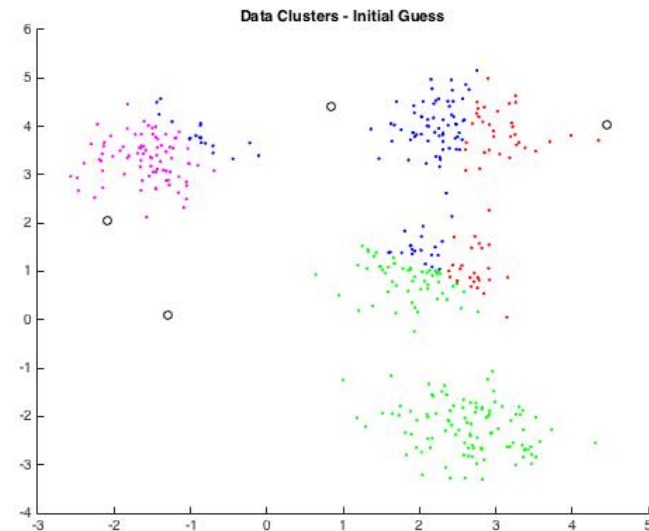
(d)



(e)



(f)



1. Initialize **cluster centroids** $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^n$ randomly.

2. Repeat until convergence: {

For every i , set

$$c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2.$$

For each j , set

$$\mu_j := \frac{\sum_{i=1}^m 1\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^m 1\{c^{(i)} = j\}}.$$

}

◆ 目标函数

◆ 类内样本围绕类均值向量的紧密程度

$$E = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|_2^2$$

◆ 收敛性

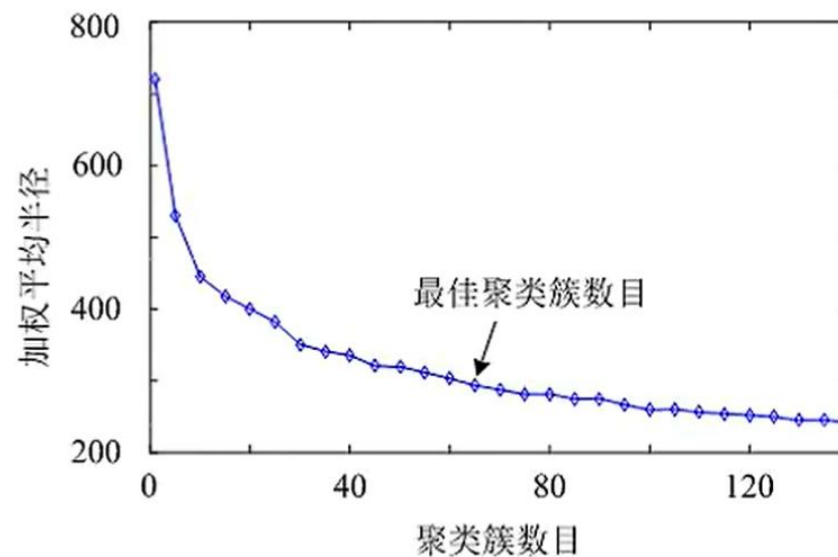
◆ NP hard问题，迭代优化：

◆ 固定均值向量，优化划分

◆ 固定划分，优化均值向量

◆ 超参选择

◆ K的个数

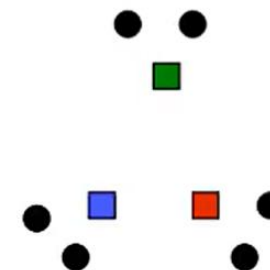
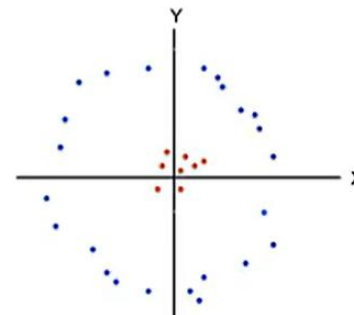
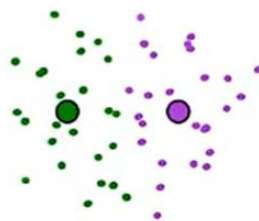


◆ 初始中心（覆盖）

- ◆ 大于最小间距的随机点/样本点
- ◆ K个相互距离最远的样本点
- ◆ K个等距网格点

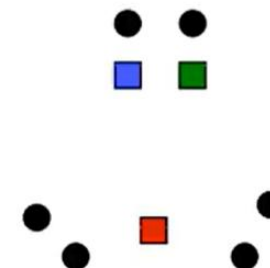
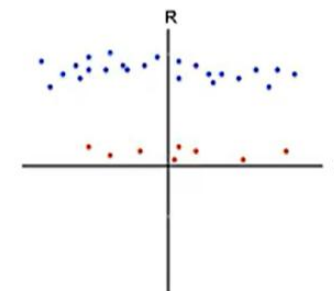
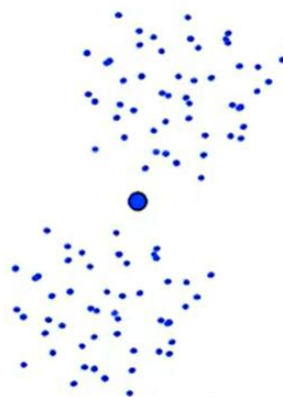
◆ 优点

- ◆ 实现简单
- ◆ 时间复杂度低 $O(N)$



◆ 缺点

- ◆ K值选择
- ◆ 初始中心影响
- ◆ 适合凸集



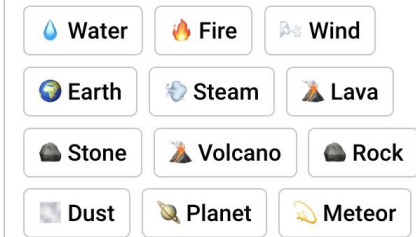
层次聚类

层次聚类

NEAL.FUN

Infinite
Craft

Items 12

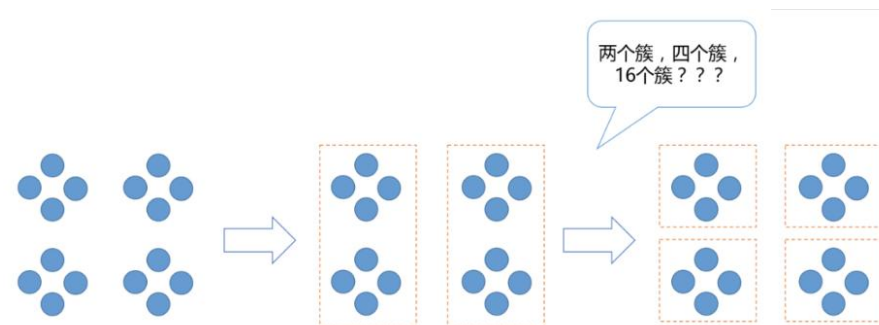


<https://neal.fun/infinite-craft/>

◆ 建立一棵层次嵌套聚类树

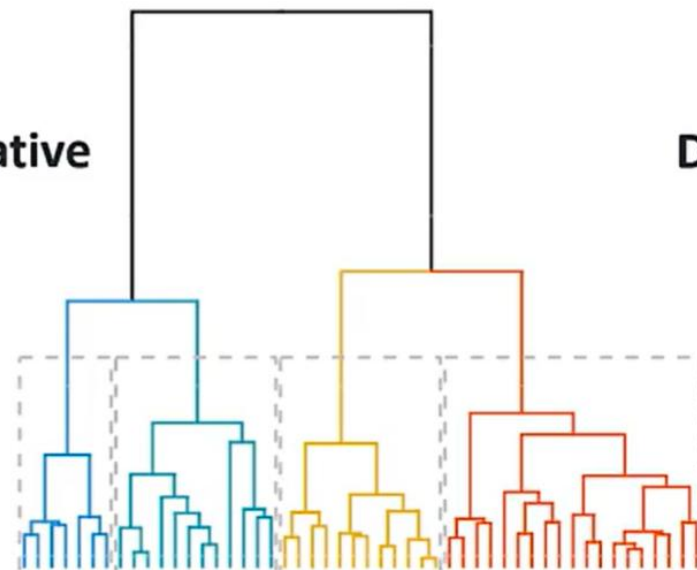
◆ 自底而上（聚合）

◆ 自顶向下（分裂）



Agglomerative

Divisive



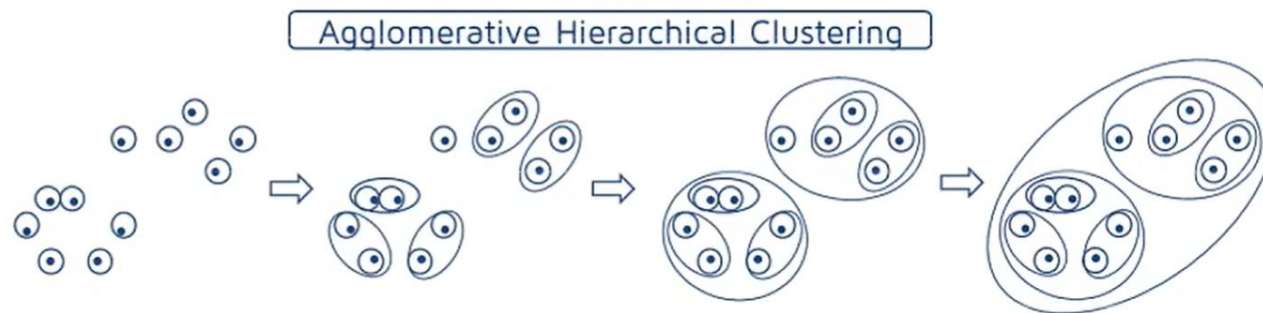
◆ 聚合

◆ 将每个样本分到单独的类

◆ 迭代:

◆ 找到距离最小的两个类

◆ 合并



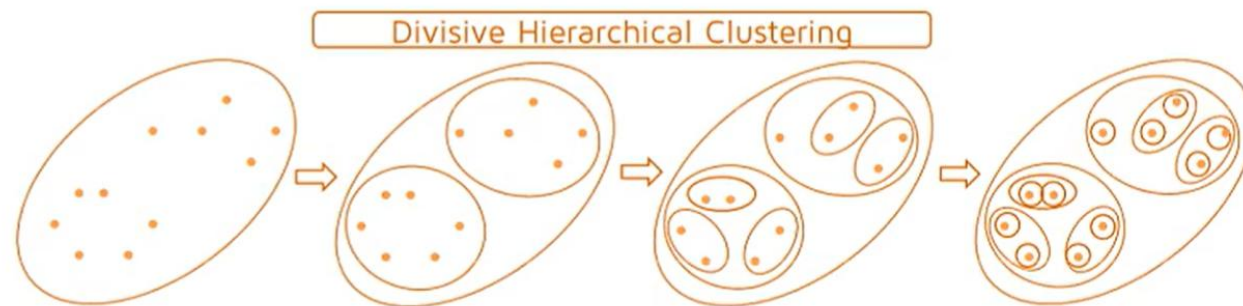
◆ 分裂

◆ 将所有样本划分到同一个类

◆ 迭代:

◆ 找到距离最远的两个样本

◆ 分配



◆ 优点

- ◆ 简单，容易理解

◆ 缺点

- ◆ 合并点/分裂点选择不太容易
- ◆ 合并/分裂的操作不能撤销
- ◆ 不适合大数据集
- ◆ 效率较低 $O(TN^2)$ ， T 为迭代次数， N 为样本点数

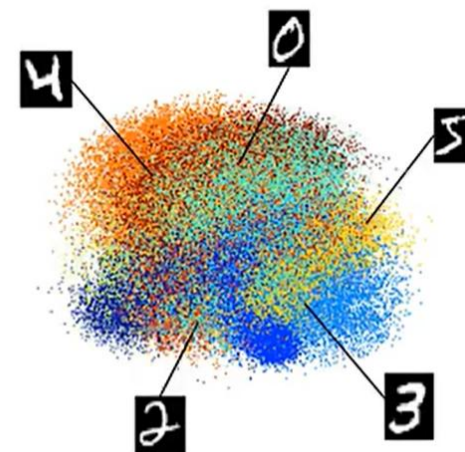
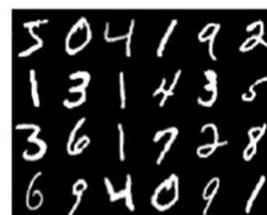
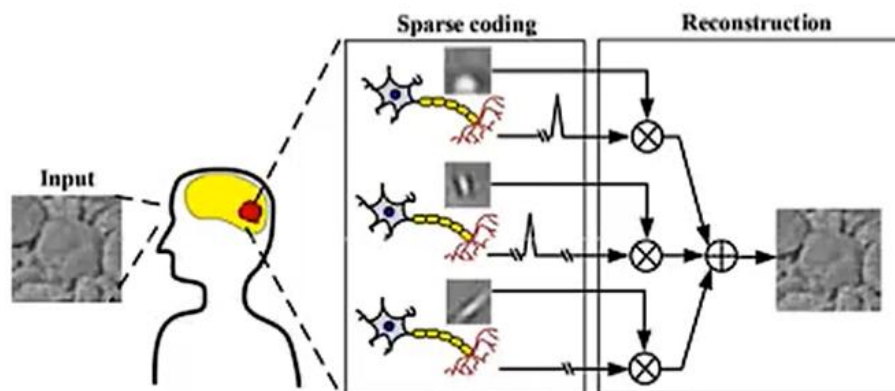
无监督特征学习

◆ 从无标签数据中学习有用的特征（表示）

◆ 应用：特征提取、去噪、降维、数据可视化

◆ 降维/可视化：

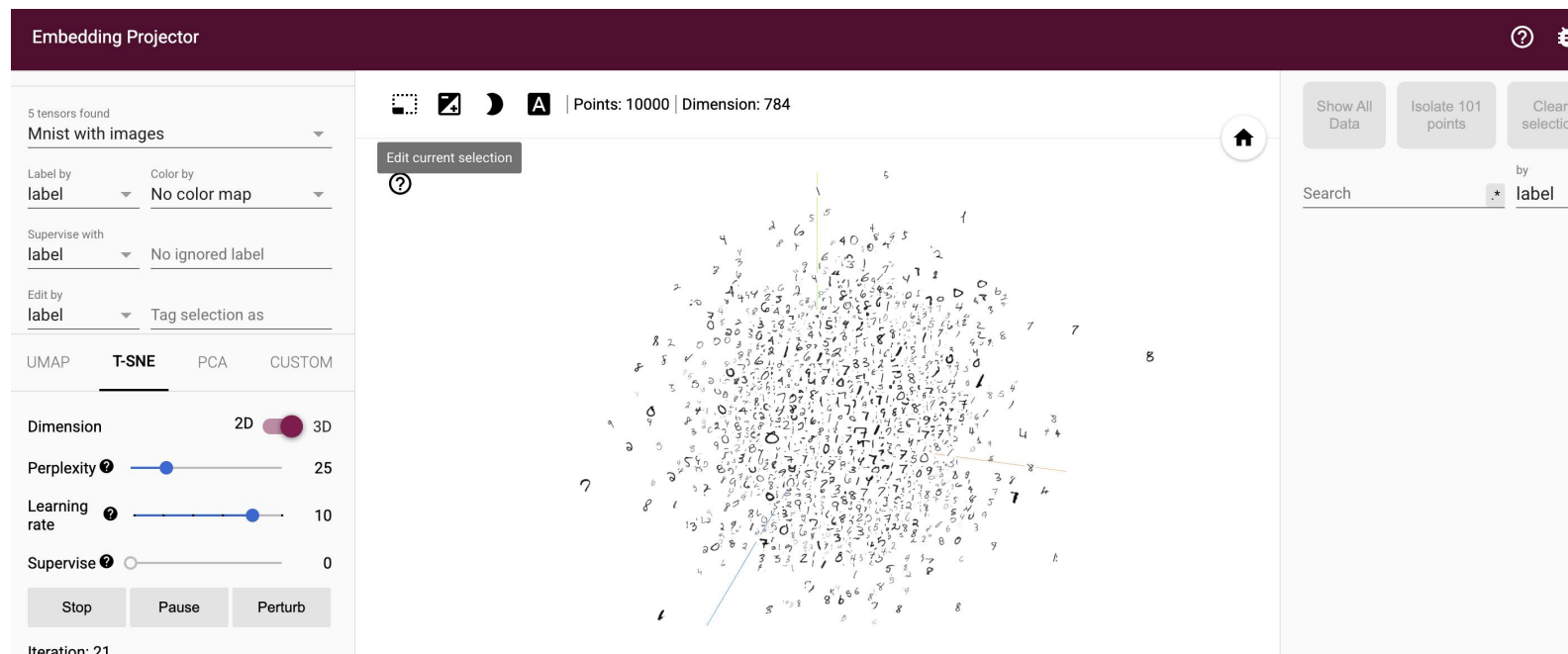
◆ 稀疏编码：



像人脑，用少量
信息来表示样本

主成分分析

主成分分析



<https://projector.tensorflow.org/>

目标：使得在转换后的空间中数据的方差最大

◆ 样本点 $\mathbf{x}^{(n)}$ 投影之后的表示为

$$z^{(n)} = \mathbf{w}^\top \mathbf{x}^{(n)}$$

◆ 所有样本投影后的方差为

$$\begin{aligned}\sigma(\mathbf{X}; \mathbf{w}) &= \frac{1}{N} \sum_{n=1}^N (\mathbf{w}^\top \mathbf{x}^{(n)} - \mathbf{w}^\top \bar{\mathbf{x}})^2 \\ &= \frac{1}{N} (\mathbf{w}^\top \mathbf{X} - \mathbf{w}^\top \bar{\mathbf{X}})(\mathbf{w}^\top \mathbf{X} - \mathbf{w}^\top \bar{\mathbf{X}})^\top \\ &= \mathbf{w}^\top \Sigma \mathbf{w},\end{aligned}$$

◆ 目标函数

$$\max_{\mathbf{w}} \mathbf{w}^\top \Sigma \mathbf{w} + \lambda(1 - \mathbf{w}^\top \mathbf{w})$$

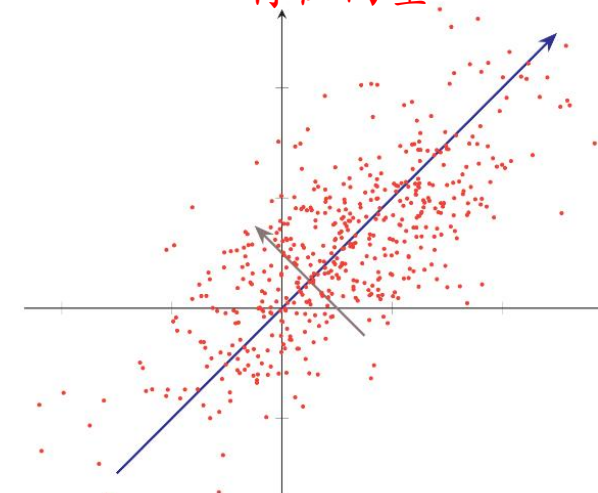
约束项

◆ 目标函数求导并等于0

$$\Sigma \mathbf{w} = \lambda \mathbf{w}$$

协方差矩阵 特征值 特征向量

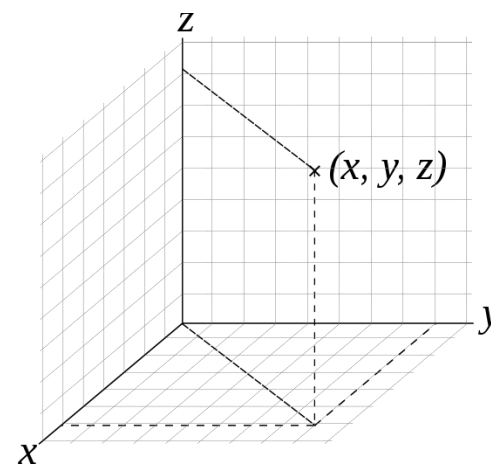
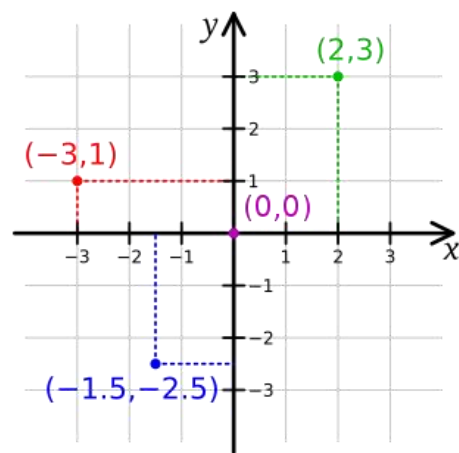
矩阵特征值
分解问题，
从大到小排列
特征值。



编码与稀疏编码

◆ 编码：对D维度空间中的样本 $\mathbf{x}$ 找到其在P维空间中的表示（或投影）。

◆ 完备性：如果M个基向量刚好可以支撑起M维的欧氏空间，则这M个基向量是完备的，如果M个基向量可以支撑起D维的欧氏空间 ($M > D$)，则这M个基向量是过完备的。



◆ 给定一组“完备”的基向量 $A = [a_1, a_2, \dots, a_m]$ ，将输入样本 x 表示为这些基向量的线性组合。

$$x = \sum_{m=1}^M z_m a_m$$

$$= Az,$$

字典

编码

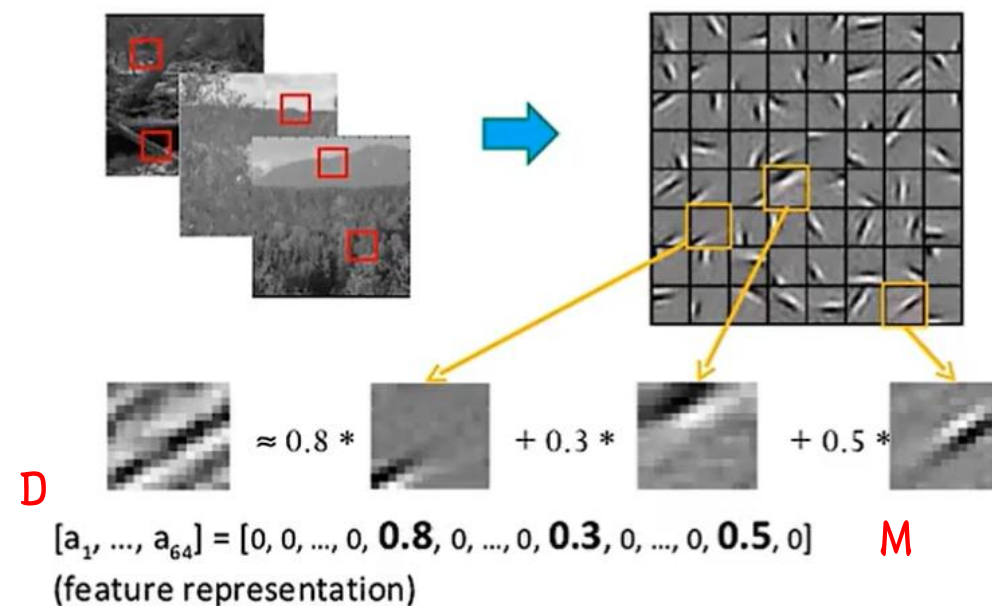
$$\begin{bmatrix} x \end{bmatrix} = \begin{bmatrix} A \end{bmatrix} \begin{bmatrix} z \end{bmatrix}$$

◆找到一组“过完备”的基向量（即 $M > D$ ）来进行编码

◆稀疏编码的目标函数定义为

$$\mathcal{L}(A, Z) = \sum_{n=1}^N \left(\left\| \mathbf{x}^{(n)} - A\mathbf{z}^{(n)} \right\|^2 + \eta \rho(\mathbf{z}^{(n)}) \right)$$

Sparse coding illustration



$\rho(\cdot)$ 是一个稀疏性衡量函数， η 是一个超参数，用来控制稀疏性的强度

◆稀疏编码的目标函数:

$$\mathcal{L}(\mathbf{A}, \mathbf{Z}) = \sum_{n=1}^N \left(\left\| \mathbf{x}^{(n)} - \mathbf{A}\mathbf{z}^{(n)} \right\|^2 + \eta \rho(\mathbf{z}^{(n)}) \right)$$

◆衡量稀疏性的函数可以选择

(非零元素的比例)

l_0 范数

$$\rho(\mathbf{z}) = \sum_{i=1}^p \mathbf{I}(|z_i| > 0)$$

不是连续可导

l_1 范数

$$\rho(\mathbf{z}) = \sum_{i=1}^p |z_i|$$

对数函数

$$\rho(\mathbf{z}) = \sum_{i=1}^p \log(1 + z_i^2)$$

指数函数

$$\rho(\mathbf{z}) = \sum_{i=1}^p -\exp(-z_i^2)$$

常用

◆ 训练过程一般用交替优化的方法进行。

1. 固定基向量，对输入计算其对应的最优编码

$$\min_{\mathbf{z}^{(n)}} \left\| \mathbf{x}^{(n)} - \mathbf{A}\mathbf{z}^{(n)} \right\|^2 + \eta \rho(\mathbf{z}^{(n)}), \forall n \in [1, N]$$

2. 固定上一步得到的编码，计算其最优基向量

$$\min_{\mathbf{A}} \sum_{n=1}^N \left(\left\| \mathbf{x}^{(n)} - \mathbf{A}\mathbf{z}^{(n)} \right\|^2 \right) + \lambda \frac{1}{2} \|\mathbf{A}\|^2$$

◆ 优点

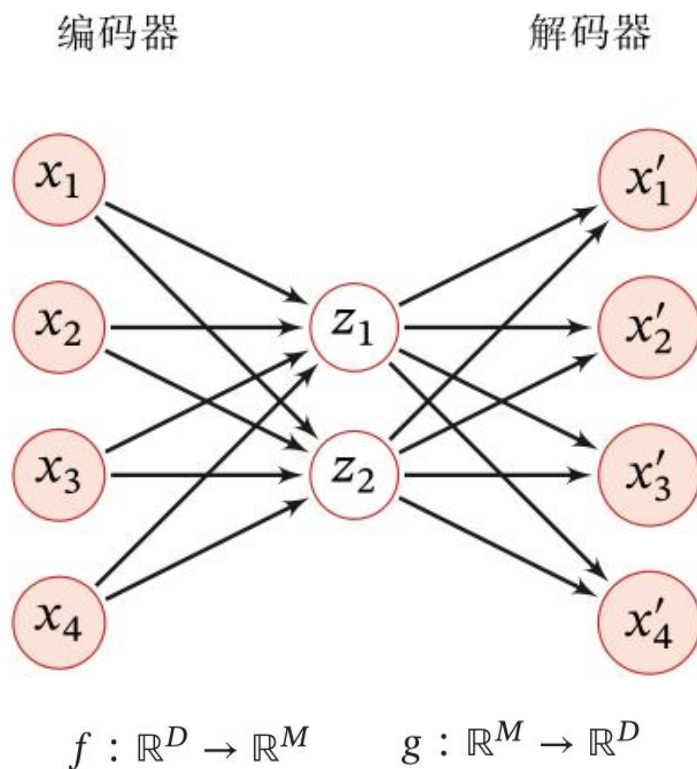
◆ 稀疏性带来的最大好处就是可以极大地降低计算量。

◆ 可解释性强。因为稀疏编码只有少数的非零元素，相当于将一个输入样本表示为少数几个相关的特征，可以更好地描述其特征，并易于理解。

◆ 可以实现特征的自动选择，只选择和输入样本相关的最少特征，从而可以更好地表示输入样本，降低噪声并减轻过拟合。

自编码器

◆ 编码器 (Encoder)—解码器 (Decoder) 架构



◆ 学习目标：最小化 **重构误差**

$$\begin{aligned}\mathcal{L} &= \sum_{n=1}^N \|\mathbf{x}^{(n)} - g(f(\mathbf{x}^{(n)}))\|^2 \\ &= \sum_{n=1}^N \|\mathbf{x}^{(n)} - f \circ g(\mathbf{x}^{(n)})\|^2.\end{aligned}$$

ConvNetJS Denoising Autoencoder demo

Description

All the other demos are examples of Supervised Learning, so in this demo I wanted to show an example of Unsupervised Learning. We are going to train an autoencoder on MNIST digits.

An autoencoder is a regression task where the network is asked to predict its input (in other words, model the identity function). Sounds simple enough, except the network has a tight bottleneck of a few neurons in the middle (in the default example only two!), forcing it to create effective representations that compress the input into a low-dimensional code that can be used by the decoder to reproduce the original input.

Report questions/bugs/suggestions to [@karpathy](#).

Training Stats

Current image:

Forward time per example: 0ms

Backprop time per example: 0ms

Regression loss: 22.27123

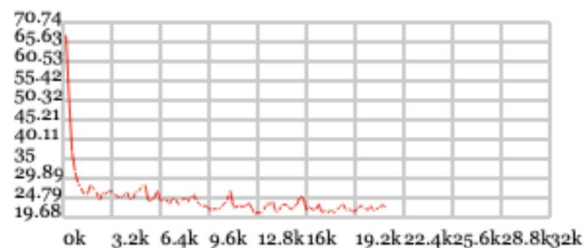
L2 Weight decay loss: 0.00915

L1 Weight decay loss: 0

Examples seen: 21346

Learning rate:

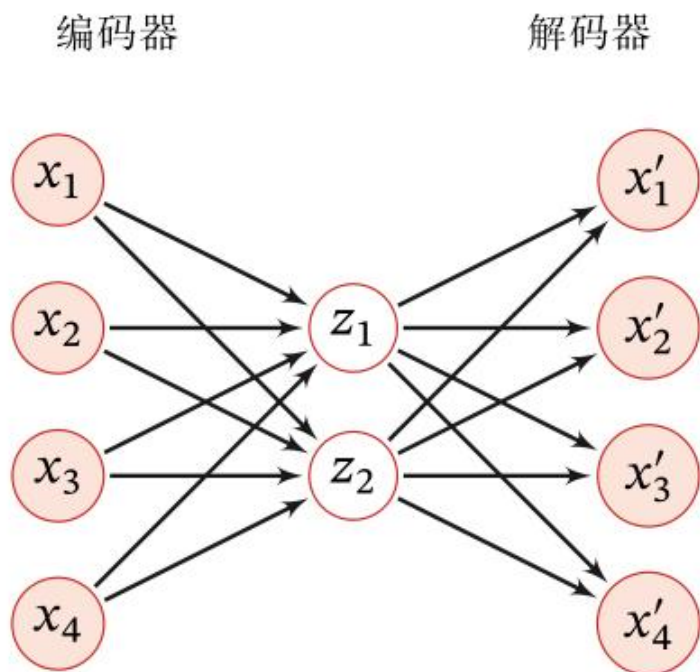
Loss:



<https://cs.stanford.edu/people/karpathy/convnetjs/demo/autencoder.html>

◆ 捆绑权重

$$\mathbf{W}^{(2)} = \mathbf{W}^{(1)\top}$$



参数更少、一定程度的正则化

$$\mathbf{z} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}),$$

$$\mathbf{x}' = f(\mathbf{W}^{(2)}\mathbf{z} + \mathbf{b}^{(2)}),$$

$$\mathcal{L} = \sum_{n=1}^N \|\mathbf{x}^{(n)} - \mathbf{x}'^{(n)}\|^2 + \lambda \|\mathbf{W}\|_F^2.$$

◆通过给自编码器中隐藏层单元加上**稀疏性限制**，自编码器可以学习到数据中一些有用的结构。

◆目标函数

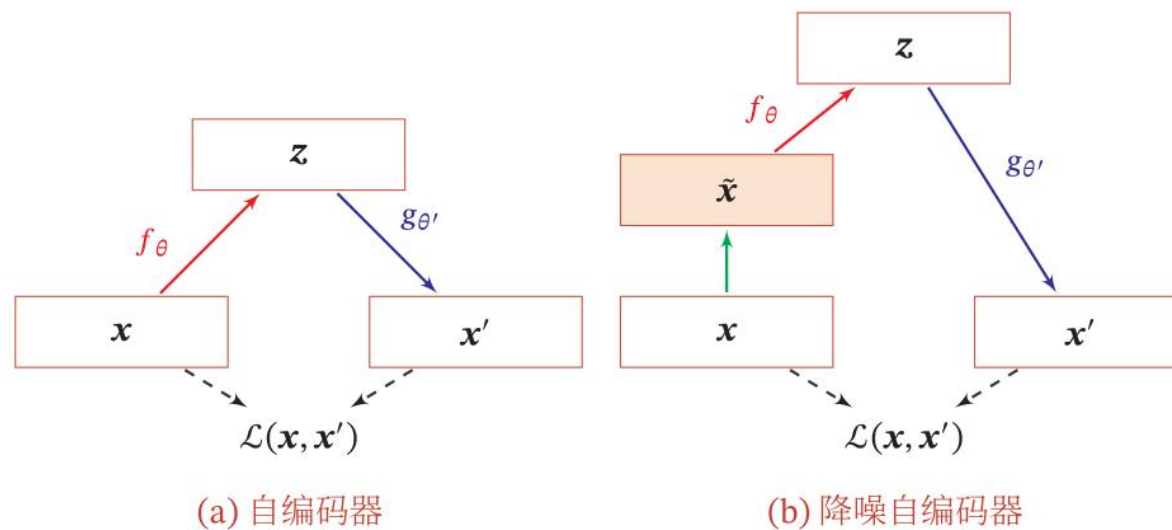
$$\mathcal{L} = \sum_{n=1}^N \|\mathbf{x}^{(n)} - \mathbf{x}'^{(n)}\|^2 + \eta \rho(\mathbf{Z}) + \lambda \|\mathbf{W}\|^2$$

◆ 降噪自编码器

◆ 通过引入噪声来增加编码鲁棒性的自编码器

◆ 对于一个向量 x ，我们首先根据一个比例 μ 随机将 x 的一些维度的值设置为0，得到一个被损坏的向量 $\tilde{x}$ 。

◆ 然后将被损坏的向量 $\tilde{x}$ 输入给自编码器得到编码 z ，并重构出原始的无损输入 x 。



◆ 建立代理任务

than local patterns. This pretext task drives the model to learn semantic concepts of objects in this way.

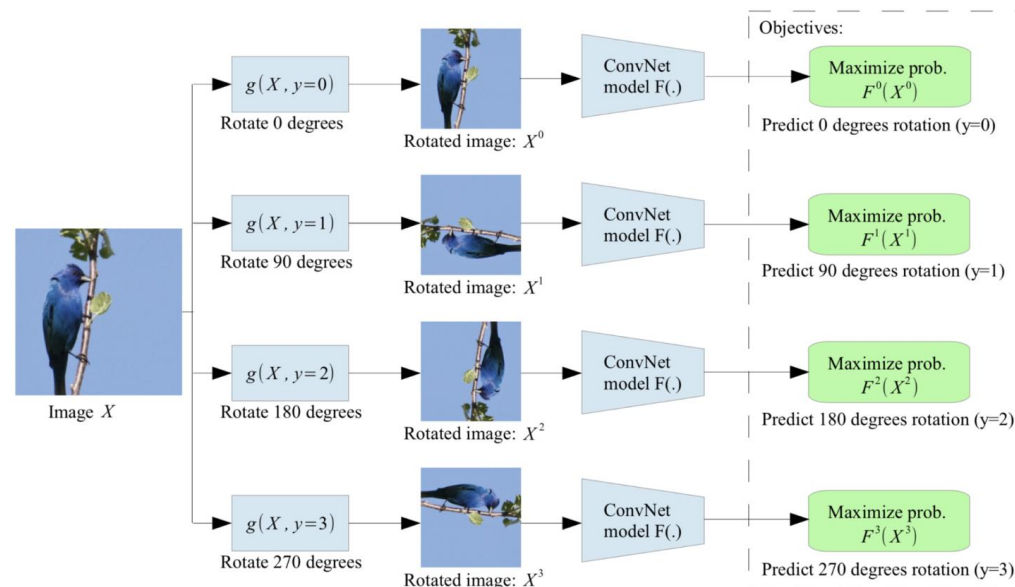


Figure 3: Illustration of self-supervised learning by rotating the entire input images. The model learns to predict which rotation is applied. (Image source: Gidaris et al. 2018)

<https://lilianweng.github.io/posts/2019-11-10-self-supervised/>

概率密度估计

◆ 参数密度估计 (Parametric Density Estimation)

- ◆ 根据先验知识假设随机变量服从某种分布，然后通过训练样本来估计分布的参数。
- ◆ 估计方法：最大似然估计

◆ 非参数密度估计 (Nonparametric Density Estimation)

- ◆ 不假设数据服从某种分布，通过将样本空间划分为不同的区域并估计每个区域的概率来近似数据的概率密度函数。

◆ 正态分布参数密度估计

◆ 假设样本 $\mathbf{x} \in \mathbb{R}^D$ 服从正态分布

$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

◆ 对数似然函数

$$\log p(\mathcal{D}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{N}{2} \log\left((2\pi)^D |\boldsymbol{\Sigma}|\right) - \frac{1}{2} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x}^{(n)} - \boldsymbol{\mu})$$

◆ 求关于 $\boldsymbol{\mu}, \boldsymbol{\Sigma}$ 的偏导，并等于0

$$\boldsymbol{\mu}^{ML} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)},$$

$$\boldsymbol{\Sigma}^{ML} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu}^{ML})(\mathbf{x}^{(n)} - \boldsymbol{\mu}^{ML})^\top.$$

◆ 多项分布参数密度估计

◆ 对数似然函数

$$\log p(\mathcal{D}|\mu) = \sum_{n=1}^N \sum_{k=1}^K x_k^{(n)} \log(\mu_k).$$

二项分布的推广

$$p(x|\mu) = \prod_{k=1}^K \mu_k^{x_k}$$

◆ 引入拉格朗日乘子

$$\max_{\mu, \lambda} \sum_{n=1}^N \sum_{k=1}^K x_k^{(n)} \log(\mu_k) + \lambda \left(\sum_{k=1}^K \mu_k - 1 \right).$$

◆ 求偏导

$$\mu_k^{ML} = \frac{m_k}{N}, \quad 1 \leq k \leq K$$

◆ 参数密度估计存在的问题

◆ 模型选择问题

◆ 实际数据的分布往往是非常复杂的，而不是简单的正态分布或多项分布。

◆ 不可观测变量问题

◆ 训练的样本只包含部分的可观测变量，还有一些非常关键的变量是无法观测的，这导致我们很难准确估计数据的真实分布。

◆ 维度灾难问题

◆ 高维数据的参数估计十分困难

◆ 随着维度的增加，估计参数所需要的样本数量也增加，在样本不足时会出现过拟合。

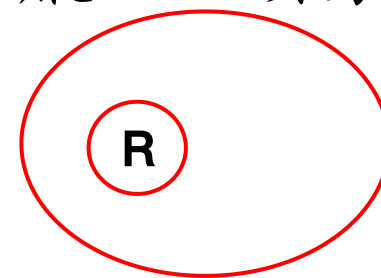
◆ 非参数密度估计

◆ 对于高维空间中的一个随机向量 $\mathbf{x}$ ，假设其服从一个未知分布 $p(\mathbf{x})$ ，则 $\mathbf{x}$ 落入空间中的小区域 R 的概率为

$$P = \int_R p(\mathbf{x}) d\mathbf{x}$$

◆ 给定 N 个训练样本，落入区域 R 的样本数量 K 服从二项分布

$$P_K = \binom{N}{K} P^K (1-P)^{1-K}$$



◆ 当 N 非常大，区域 R 足够小时，则有

$$P \approx \frac{K}{N}$$

$$P \approx p(\mathbf{x})V$$



$$p(\mathbf{x}) \approx \frac{K}{NV}$$

→ R 的体积

◆ 非参数密度估计

◆ 常用的方式:

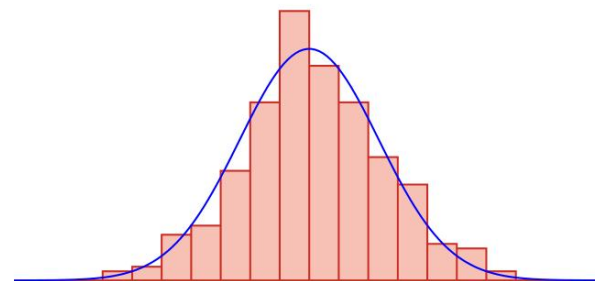
◆ 固定区域大小 V ，统计落入不同区域的数量（直方图方法、核方法）。

◆ 改变区域大小使得落入每个区域的样本数量为 K （ K 近邻方法）。

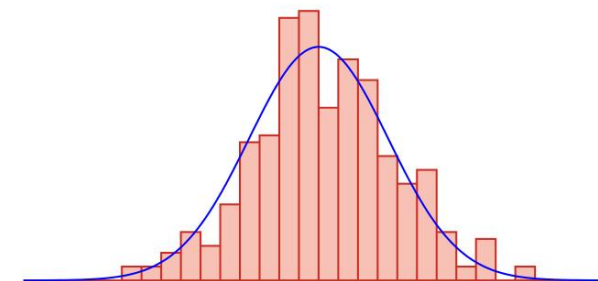
◆ 非参数密度估计

◆ 直方图方法：一种直观的估计连续变量密度函数的方法，可以表示为一种柱状图。以一维随机变量为例：

1. 将其取值范围分成 M 个连续不重叠的区间
2. 每个区间的宽度为 Δm
3. 给定 N 个训练样本统计这些样本落入每个区间的数量
4. 归一化为密度函数



(a) 10 个区间 (bin)

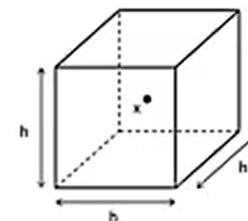
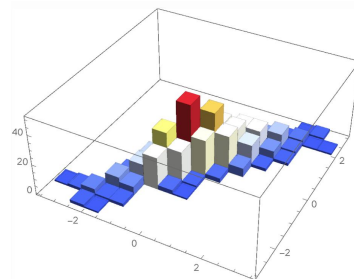


(b) 30 个区间 (bin)

$\Delta m = ?$
维度 D 增加?

◆核密度估计 (Kernel Density Estimation)

◆一种直方图方法的改进



◆假设 $\mathbf{R}$ 为 D 维空间中的一个以点 $\mathbf{x}$ 为中心的“超立方体”，并定义核函数来表示一个样本 $\mathbf{z}$ 是否落入该超立方体中

$$\phi\left(\frac{\mathbf{z} - \mathbf{x}}{h}\right) = \begin{cases} 1 & \text{if } |z_i - x_i| < \frac{H}{2}, 1 \leq i \leq D \\ 0 & \text{else} \end{cases}$$

◆点 $\mathbf{x}$ 的密度估计为

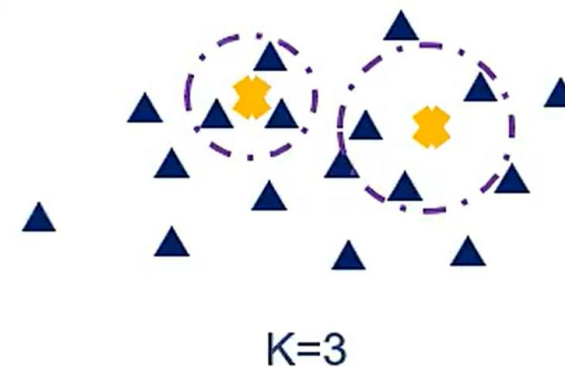
$$p(\mathbf{x}) = \frac{K}{NH^D} = \frac{1}{NH^D} \sum_{n=1}^N \phi\left(\frac{\mathbf{x}^{(n)} - \mathbf{x}}{H}\right)$$

◆K近邻方法

◆找到一个以点 $\mathbf{x}$ 为中心的球体，使得落入球体的样本数量为K

◆估算点 $\mathbf{x}$ 的密度

$$p(\mathbf{x}) \approx \frac{K}{NV}$$



- ◆核密度估计方法中的核宽度是固定的，因此同一个宽度可能对高密度区域过大，而对低密度区域过小。
- ◆K近邻方法是一种相对灵活的方法，但K值的选择十分关键，如果 K 太小，无法有效地估计密度函数；而 K 太大也会使得局部的密度不准确，并且增加计算开销。
- ◆K近邻方法也可用于分类问题

- ◆ 非参数密度估计(直方图方法除外)需要保留整个训练集
- ◆ 参数密度估计不需要保留整个训练集，因此在存储和计算上更加高效。

- ◆ **截止时间：** 5月17日晚12:00
- ◆ **提交方式：** 每人作业overleaf填写，以 pdf 格式提交，命名为“学号_姓名”，班长收齐将全班作业压缩包发送到邮箱：514034538@qq.com
 - ◆ 设计聚类k-MEANS在线可视化交互网页/平台等，操作聚类中心的影响。
 - ◆ 主成分分析在线可视化交互网页/平台等，例如选择不同主成分
 - ◆ 稀疏编码在线可视化交互网页/平台等，例如不同稀疏度
- ◆ **作业晚交政策：** 作业超过截止时间但在24小时以内，该次作业按照60%分数计算，超过24小时不计分

习题 9-1 分析主成分分析为什么具有数据降噪能力？

- ◆ **截止时间：** 5月8日晚9:00
- ◆ **提交方式：** 每人作业以 pdf 格式提交，命名为“学号_姓名”，班长收齐将全班作业压缩包发送到邮箱：meao_wangxi@163.com
- ◆ **作业晚交政策：** 作业超过截止时间但在24小时以内，该次作业按照60%分数计算，超过24小时不计分

习题 9-1 分析主成分分析为什么具有数据降噪能力？